Page Checks — Technical Reference

An engineering-oriented reference for the checks in **Is It Agent-Ready? for SitecoreAI**: the scoring model, every check with its thresholds and credit, and where each one lives in the code. The Page Builder panel ("Is this page agent-ready?") and the dashboard ("Is this site agent-ready?") use **the same engine**, so a page scores the same in both. Dashboard specifics (roll-up, cache, site checks card) are in `PAGE-MATRIX-TECHNICAL.md`.

Source of truth: `src/lib/checks/` — `registry.ts` (the check list), `page/*.ts` and `site/*.ts` (the checks), `scoring.ts` (pool maths), `gate.ts`, `engine.ts` (entry points), `context.ts` (inputs), `json-ld.ts` (shared JSON-LD parser), `adapters.ts` (the pre-v2 UI). Plan reference: `docs/IS-IT-AGENT-READY-PLAN.md` section 5.

Scoring model

- Every check belongs to one of **four categories**, phrased as questions: **Reach** (Can AI agents get to it?), **Read** (Can they read it?), **Understand** (Do they understand it?), **Trust** (Will they trust and cite it?).
- Every check has a **tier**. **Essential** checks share an **80-point pool**, **Recommended** checks a **20-point pool**, and **Bonus** checks add **up to +5**. The total is capped at 100.
- A check returns a **credit from 0 to 1** (partial credit is normal) and a status (`pass` / `warning` / `fail` / `info`). Its points are `credit × its share of the tier pool`. Shares are equal unless a check sets the optional `weight` multiplier (default 1; none are set today).
- A check can also return **N/A** (`not-applicable`) or **not checked** (`not-checked`: AI pending, PageSpeed not run, fetch failed). Both are **left out of the pools, never failed**. The pool is split across the checks that are scored, so a page is only graded on what applies to it.
- If Essential or Recommended has nothing scored, the other one is scaled up so the base still totals 100. With nothing scored at all, there is no score yet (`null`).
- The score is rounded to a whole number. Exact points are kept per check and per category (`ScoreBreakdown` in `types.ts`).
- Thresholds are shared by the panel and the dashboard (`src/lib/score-thresholds.ts`): **Agent-ready ≥ 80**, **Almost there 60–79**, **Not ready < 60**.

Points in Phase 3. Four Essential page checks exist (P2, P8, P9, P10), so when all apply each is worth 20 points; on the home page P10 is usually N/A and the other three are worth 26.7 each. The Recommended checks share 20 points (about 1.7 each when all twelve apply). Phases 4 and 7 add P3 and P5 to Essential, which brings each Essential check to about 13 points.

Scopes. Page checks run per page in the panel and in dashboard scans. Site checks run once per site and get **their own site score**; they are never part of a page score. Phase 3 has two site checks: S2 and S7.

Same score in both views. Both views build the check context with `buildPageContext` (`context.ts`) from the same inputs: the page's live HTML (`/api/preview-fetch` → `ParsedHtml`), the AI results (`/api/ai-check`, cached by content hash), and the same Sitecore fields read by the same names (`SCORING_FIELD_NAMES`: title, meta description, `robotsMetaTag`, `__Updated`). The panel reads them with `pageFieldsFromFieldMap`, the dashboard with `pageFieldsFromValues`. The one difference is PageSpeed: only the panel can run it, so after you run it there, the panel includes P7 and the dashboard doesn't. `src/lib/checks/parity.test.ts` covers this.

The gate — P1 `reach.indexable`

Location: `gate.ts`

Runs before any check. The page is **blocked** when:

- the Sitecore **Robots Meta Tag** field (`robotsMetaTag`) contains `noindex`, **or**
- the rendered `<meta name="robots">` or `<meta name="googlebot">` contains `noindex`, `nosnippet` or `max-snippet:0`.

A blocked page shows the gate message instead of a score (`PageReport.score` is `null`, no outcomes). The dashboard reads the same field (it was added to its page query), shows the row as "Blocked (noindex)", and leaves it out of the averages.

Page checks

All page checks except P7, P9, P16 and P21 read the page's **live HTML**. "Not yet scanned" (not checked) means the HTML hasn't arrived yet.

Reach

ID	Title	Tier	Credit	Was
P2 `reach.canonical`	Canonical points to this page	Essential	self-referencing `<link rel="canonical">` → 1 · points elsewhere → 0.4 · missing → 0	`seo-canonical`

Read

ID	Title	Tier	Credit	Was
P4 `read.landmarks`	Page structure is machine-readable	Recommended	landmarks present ÷ 4 (`<main>` , `<article>` , `<nav>` , `<header>`); unlabelled buttons/links are listed	`cq-agentic-readiness`
P7 `read.speed`	Page loads fast	Recommended	PageSpeed ≥ 70 → 1 · 50–69 → 0.6 · < 50 or not crawlable → 0. **Not checked** until PageSpeed runs, or when PSI errors	`cq-page-experience`

P3 (content without JavaScript), P5 and P6 (Markdown for agents) arrive in Phases 4 and 7.

Understand

ID	Title	Tier	Credit	Was
P8 `understand.headings`	One clear H1 and logical headings	Essential	half for exactly one H1 (1/6 for several), half for no skipped levels	`seo-headings`
P9 `understand.title-meta`	Title and description are set	Essential	average of title (50–60 chars → 1 · 30–70 → 0.6 · other → 0.25 · missing → 0) and description (120–160 → 1 · other → 0.5 · missing → 0). Reads the Sitecore field first, the rendered tag as fallback	`seo-meta-title` + `seo-meta-description`
P10 `understand.json-ld`	Structured data is present and valid	Essential	mean over the schemas this page needs: valid → 1 · present but missing required properties → 0.5 · missing → 0. A block that doesn't parse caps it at 0.5 (fail). N/A when nothing is expected	`aeo-structured-data` + `aeo-faq-schema`
P11 `understand.metadata`	Page identity metadata is complete	Recommended	present ÷ 5 of `html lang`, `og:type`, `og:title`, `og:description`, `og:image`	`seo-og-tags`
P12 `understand.faq-quality`	FAQ answers are quotable	Recommended	answers of 40–300 chars ÷ all answers. N/A without Q&A pairs	`aeo-faq-answer-quality`
P13 `understand.question-headings`	Sections answer real questions	Recommended	≥ 3 question H2s or ≥ 50% of H2s → 1 · at least one → 0.5 · none → 0. N/A without H2s	`aeo-question-headings` (was off)

ID	Title	Tier	Credit	Was
P14 `understand.readability`	Easy to read and quote	Recommended	half: share of 20+-word paragraphs at ARI grade ≤ 13; half: average paragraph 40–120 words → 1 · 20–200 → 0.5. N/A without paragraphs	`cq-reading-grade` + the matrix's paragraph length
P15 `understand.image-alt`	Images are described	Recommended	images with alt ÷ all images. N/A without images	`seo-image-alt`

P10 in detail (fixes D2). The three old schema checks always passed and only printed a snippet. P10 reads the real `application/ld+json` blocks through the shared parser and expects:

- **BreadcrumbList** on every page except the home page. Required: `itemListElement` with `position` and `name` on each item.
- **FAQPage** when the page has two or more Q&A pairs. Required: `mainEntity` with `name` and `acceptedAnswer.text` on each question.
- **Article** (or any subtype, e.g. `BlogPosting`, `NewsArticle`) on article templates (template name "Article Detail Page"). Required: `headline` (or `name`).

The fix snippet contains only the missing or incomplete schemas (`buildBreadcrumbJsonLd`, `buildFaqPageJsonLd`, `buildArticleJsonLd` in `helpers.ts`). The site's Organization schema is S7, not part of P10.

Q&A pairs (P10, P12). `extractQaPairs` (`src/lib/server/extract-html.ts`) finds them in **any component**: an accordion toggle (`[aria-controls][aria-expanded]`) whose controlled panel holds the answer (collapsed, `aria-hidden` panels included), or a `<details>` whose `<summary>` is the question. Only toggles that read as a question (end in "?" or start with a question word) count, so menus and "Expand all" are skipped. On BrandX's FAQ page it finds all 8 questions. The AccordionItem datasource fetch in `usePageFields` is no longer used for scoring.

Trust

ID	Title	Tier	Credit	Was
P16 `trust.uniqueness`	Content is original, not generic	Recommended	AI rating `unique` → 1 · `borderline` → 0.5 · `commodity` → 0. Not checked until the AI has run	`cq-uniqueness`
P17 `trust.author`	Author is named and linked	Recommended	named and linked → 1 · named only → 0.5 · none → 0. **Article templates only** (N/A elsewhere)	`cq-author` (was off)
P18 `trust.freshness`	Content is recently updated	Recommended	≤ 180 days → 1 · ≤ 365 → 0.6 · ≤ 730 → 0.25 · older or no date → 0	`cq-freshness` (was off) + the matrix's bands
P19 `trust.depth`	Enough depth to be cited	Recommended	words in body paragraphs: ≥ 800 → 1 · ≥ 400 → 2/3 · ≥ 150 → 1/3 · fewer → 0	the matrix's content depth
P20 `trust.anchor-text`	Links say where they go	Recommended	generic anchors ≤ 20% → 1 · ≤ 40% → 0.5 · more → 0. N/A without links	`seo-anchor-text` (was info only)
P21 `trust.follow-up-gaps`	Answers the next question too	**Bonus**	follow-up questions covered ÷ all. Not checked until the AI has run	`cq-followup-gaps` (was off)

- **P16 body and caching.** The body is `ParsedHtml.mainText` from `/api/preview-fetch`: the text of `<main>` (or `<body>` without header, nav and footer), one block per line, entities decoded, capped at 20,000 chars; `/api/ai-check` sends the first 3,000 to Claude on `ANTHROPIC_MODEL_BULK` (effort `low`, structured output). The panel calls `ai-check` after `preview-fetch` returns and skips it when there's no main text (console: `[scan] ai-check body: …`). Results are keyed by SHA-256 of the normalised title and text (`src/lib/ai/content-key.ts`) and cached in localStorage (`aiDiscovery:ai-results:<AI_CHECK_VERSION>`, shared by the panel and the dashboard) and in memory on the server, so an unchanged page doesn't call Claude again (console: `[ai-cache] hit …`). The AI re-run button skips both caches. A refusal returns `422` and the check stays not checked. Phase 4 switches the body to the sanitised `getPageHTML` draft.
- **P17 author signals**, in order: the `author` of an Article-type JSON-LD node (a string, an inline Person/Organization, or an `@id` reference; linked when it has `url` or `sameAs`), then `<meta name="author">`, then a `rel="author"` link (which counts as the profile link).

- **P18 date sources**, in order: `<meta property="article:modified_time">`, a JSON-LD `dateModified`, `<meta name="last-modified">`, then Sitecore `__Updated`. `parseDate` (`helpers.ts`) reads ISO dates and Sitecore's compact `yyyyMMddTHHmmssZ` format.
- **P20 generic anchors.** `isGenericAnchor` (`helpers.ts`) matches the `ANCHOR_BLOCKLIST` exactly, plus a prefix rule limited to phrases that stay generic whatever follows ("click here …", "read more …", "see more …"). Descriptive anchors that start with a blocklisted word ("Download the 2026 report", "Register for the webinar") pass (SPIKE-FINDINGS §2).

`aeo-howto-schema` stays off (plan section 5.3); revisit only if HowTo content templates exist.

Site checks

Run once per site by `runSiteChecks` (`engine.ts`), with their own site score from the same pool maths. The dashboard shows them in the **Site checks** card on the Overview (Phase 6 moves them to the Agent access tab).

ID	Title	Tier	Credit	Was
S2 `reach.robots-ai-rules`	robots.txt lets AI search in	Essential	0 of 6 named AI crawlers blocked → 1 · 1–2 → 0.5 · 3+ → 0	the matrix's crawler permissions, moved as is; Phase 6 replaces it with bot groups by purpose
S7 `understand.org-schema`	Your organisation is described for AI	Recommended	see below	`aeo-organization-schema`

S7 `understand.org-schema` (`site/org-schema.ts`)

Graded on the **home page's live HTML** (the `getPageHTML` draft has no `<head>` scripts). The four rules agreed on 2026-10-31:

What the home page has	Result
Complete Organization schema	credit 1, pass
Organization schema with missing fields	partial credit, warning listing the fields, corrected snippet
No Organization schema (page fetched)	credit 0, **warning** ("Not added yet"), ready-to-paste snippet
Home page couldn't be fetched	**N/A** ("Couldn't check"), snippet

- **Credit:** 70% for the required fields (`name` ; `url` on the site's own domain, `www.` and subdomains allowed; `logo` as a URL or ImageObject) and 30% for the recommended ones (`sameAs` , `description` , `contactPoint`). Bonus fields (`address` as a PostalAddress, `legalName`) are listed as tips and not scored.
- **Detection:** every `application/ld+json` block, including `@graph` bundles, `@type` arrays (`["Organization", "LocalBusiness"]`), all 167 Organization subtypes from schema.org V30.1 (`schema-org-types.ts` , generated by `scripts/gen-schema-org-types.mjs`), and a `WebSite.publisher` that points to an Organization inline or by `@id` . The same entity declared twice under one `@id` counts once.
- **A block that doesn't parse** is a fail (credit 0) with the parse error shown.
- **Config is the expected value:** when `NEXT_PUBLIC_ORG_NAME` is set and the schema names a different organisation, or there are several Organization entries with conflicting names or URLs, the result is a warning and the credit is capped at 0.9.
- **Fix snippet:** starts from the schema already on the site and fills only the gaps (name, url, logo, description, sameAs) from `NEXT_PUBLIC_ORG_*` (`getOrganizationInfo` , with the site's name and domain as fallbacks). A `url` off the site's domain is replaced; other values are kept. With no schema at all it's `buildOrganizationJsonLd` .
- **In the panel** S7 shows only when the home page is open, in a "Site check — not in this page's score" section. The "site issues affecting this page" strip comes in Phase 6.
- **Evidence:** the BrandX home page has no JSON-LD at all (checked 2026-10-31 and 2026-11-01). The old check gave it 5/5; S7 gives it 0 with a warning and a snippet.

Shared JSON-LD parser (`json-ld.ts`)

`extract-html.ts` keeps every block as `ParsedHtml.jsonLdScripts` , either `{ ok: true, value }` or `{ ok: false, error }` , so parse errors are reported instead of dropped. `collectNodes` turns the blocks into typed nodes: each top-level object, each array entry and each `@graph` member (nested objects stay inside their parent; read them with `resolve` , which also follows `{"@id": …}` references). `@type` is always a list with `schema:` / `https://schema.org/` prefixes removed. `isOrganization` and `isArticle` match subtypes. `typesOnPage` feeds the citation forecast prompt.

The adapter to the pre-v2 UI

`adapters.ts` maps reports onto the current components until the Phase 5 redesign:

- **Panel:** `rowsByCategory` → one `CategorySection` per category (`src/components/CategorySection.tsx`) with a `RuleRow` per check showing `points / max` (one decimal), "n/a", "not checked" or a **Run scan** button for AI and PageSpeed checks, and "+N pts" next to the fix. N/A checks are listed under "Doesn't apply here". The header ring shows the 0–100 score.
- **Dashboard:** `summarisePageReport` → `PageScoreSummary` (`score`, `label`, `blocked`, `gateReason`, per-category earned/max), which is what the dashboard caches.

Meta & social generator

`src/components/MetaGenerator.tsx` → `POST /api/generate-meta` (`{ field, pageTitle, bodyText, currentValue }` → `{ value, maxChars, overLimit }`), then `xmc.pages.updateFields` on **Populate field**. The logic is in `src/lib/server/generate-meta.ts`.

- **Model:** `ANTHROPIC_MODEL` (default `claude-opus-5-5`), effort `low`, structured output `{ value }`.
- **Length limits** (title 60, description 155, OG title 70, OG description 200): structured outputs can't enforce a maximum length, so the route checks it. An overrun gets one retry that tells the model the count ("62 characters, over the 60-character limit"). If the retry is still too long, it returns the value with `overLimit: true`, and the UI shows the count in amber. Values are never cut off.
- **Prompt caching:** the brand voice (system prompt) and the page title and text (first user block) carry `cache_control`, and the field instruction comes after them. Generating the other fields, or regenerating within 5 minutes, reads the cached prefix. All four fields use one output schema, so the cache is shared. The server log shows `cache_read=N` on each `[anthropic] generate-meta: <field>` line.
- **Body:** `bodyText` is the same `ParsedHtml.mainText` the uniqueness check uses. **Generate** is disabled while the scan is still reading it. If the scan finishes without text, the panel says so and generation falls back to the title and current values. The prompt tells the model to write from the page content and not copy the current value.
- **Brand voice:** `getBrandVoice()` (`src/lib/brand/brand-voice.ts`, server only) reads the Markdown file named by `BRAND_VOICE_FILE` (relative to the project root, read once per server instance) and the name in `BRAND_NAME`. With neither set, or if the file can't be read, it uses a neutral default voice. `brand/brandx.md` is the BrandX demo voice, and `next.config.ts` (`outputFileTracingIncludes`) bundles `brand/` into the Vercel function.
- **Log:** `[generate-meta] <field>: N chars of body text, brand voice: <source>`.

Server-side fetching (SSRF protection)

Every server route that fetches a caller-supplied URL (`preview-fetch` , `robots-check` , `footer-check` , `citation-forecast` , and `psi-check` for its Google call) uses `safeFetch` from `src/lib/server/safe-fetch.ts` :

- `http` / `https` only; no credentials in the URL.
- Blocks private, loopback, link-local (incl. `169.254.169.254`), CGNAT, multicast, reserved and documentation ranges, IPv4 and IPv6, including IPv4-mapped and NAT64/6to4/Teredo forms. The check runs in the undici Agent's DNS lookup at connect time, so DNS rebinding can't bypass it. IP-literal hosts (including `http://2130706433/` -style encodings) are checked before the request.
- Follows at most 3 redirects (re-checked each hop) and drops the caller's headers, such as the editing token, on a cross-origin hop.
- One timeout across redirects and body; bodies capped at 5 MB by default.
- Blocked input → `SafeFetchError` with `isBadInput` , and the route answers **400** `{ error: "blocked_url" }` . Fetch failures stay **502**, or the route's own fallback.
- `SAFE_FETCH_ALLOW_HOSTS` (comma-separated) skips the address check for named hosts. Use it only for local development.
- Tests: `src/lib/server/safe-fetch.test.ts` .

Tests

File	Covers
`src/lib/checks/scoring.test.ts`	pools, partial credit, weights, N/A and not checked, rescaling, bonus cap, categories, the gate
`src/lib/checks/page/page-checks.test.ts`	P10 real detection (D2), P9, N/A and not-checked cases, P17, P18, the fixture page
`src/lib/checks/site/org-schema.test.ts`	S7: the four rules, `@graph`, `@type` arrays, subtypes, `WebSite.publisher`, parse errors, config mismatch, duplicates, BrandX
`src/lib/checks/json-ld.test.ts`	the shared parser
`src/lib/checks/parity.test.ts`	panel and dashboard read the same fields and give the same score; the 80 / 60 thresholds
`src/lib/server/extract-html.test.ts`	`mainText`, Q&A pairs (BrandX accordion markup), metadata, JSON-LD blocks

Key files

File	Responsibility
`src/lib/checks/registry.ts`	Every check, in display order
`src/lib/checks/page/{reach,read,understand,trust}.ts`	Page checks P2–P21
`src/lib/checks/site/reach.ts`, `site/org-schema.ts`	Site checks S2, S7
`src/lib/checks/scoring.ts`, `gate.ts`, `engine.ts`	Pool maths, the gate, `runPageChecks` / `runSiteChecks`
`src/lib/checks/context.ts`	`buildPageContext`, the shared field names, article/home detection
`src/lib/checks/json-ld.ts`, `schema-org-types.ts`	JSON-LD parsing and the schema.org subtype lists
`src/lib/checks/adapters.ts`	Reports → pre-v2 UI rows and dashboard summaries
`src/lib/rule-engine/helpers.ts`	ARI, word counts, anchor blocklist, dates, snippet builders
`src/lib/rule-engine/types.ts`	Input types (`ParsedHtml`, `AiResults`, `PageData`, …)
`src/lib/rule-engine/test-fixture.ts`	Demo data for `?demo=1`
`src/lib/score-thresholds.ts`	Shared 80 / 60 thresholds
`src/lib/server/extract-html.ts`	Parses fetched HTML into `ParsedHtml`
`src/lib/brand/organization.ts`, `brand-voice.ts`	Organization identity and brand voice config